KDA 技术路线：从 Delta Rule 到 Kimi K3

Kimi K3 中的 Kimi Delta Attention，简称 KDA。

如果直接看 KDA 的最终公式，很容易被一堆矩阵乘法绕进去。按论文引用倒推，真正**一步接一步**的理论线是：

Linear Attention（2020）→ DeltaNet 思想（2021）→ DeltaNet 工程化（2024）→ Gated DeltaNet（2025）→ KDA（2025）→ K3（2026）。

1. Delta Rule：根据误差更新模型参数

- 论文：Bernard Widrow、Marcian E. Hoff，Adaptive Switching Circuits（1960）
- 链接：https://isl.stanford.edu/~widrow/papers/c1960adaptiveswitching.pdf

早在 1960 年，Widrow-Hoff 提出了 Delta Rule。

在此之前，更流行的直觉来自 Hebbian learning。Hebb 在 1949 年提出：**一起发放的神经元，连接会加强**——这是解释大脑突触怎么变强的神经科学假说，不是当时工程里直接拿来训练分类器的算法。后来教材把它写成权重更新，常见形式是

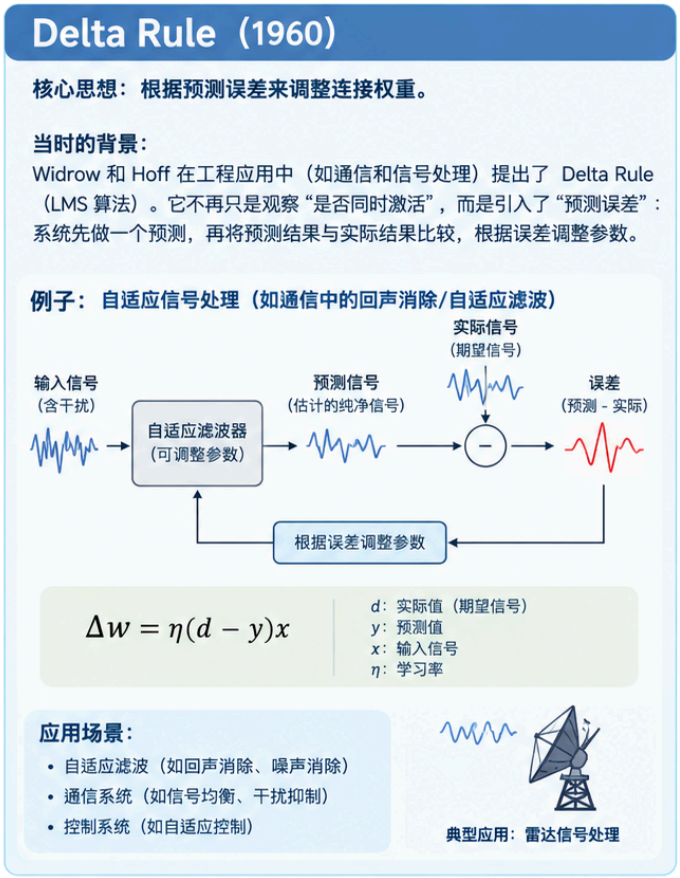
$$\Delta \boldsymbol{w} = \eta\, y\, \boldsymbol{x}$$

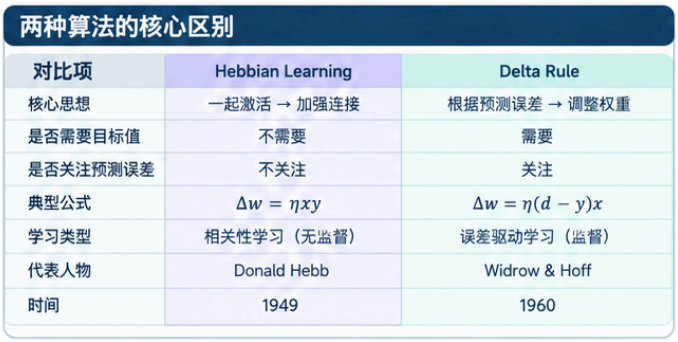
这里只有输入 $\boldsymbol{x}$ 和模型输出 y，**没有真实标签** t。连接加强多少，主要取决于输入和输出是否同时活跃。

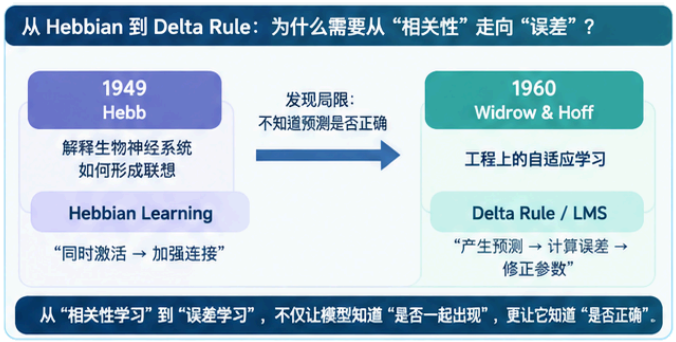
Delta Rule 把 **预测误差** 放进更新式，先算 $t - y$，再决定权重沿 $\boldsymbol{x}$ 方向改多少。式子里的 $\Delta = t - y$，是 delta 第一次作为驱动更新的核心量。

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\,(t - y)\,\boldsymbol{x}$$









2. Linear Attention：因果解码与 RNN 递推

论文：Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention（Katharopoulos 等，ICML 2020）

线性注意力的奠基工作：把自回归注意力里的 softmax 写成标准核函数形式，再用矩阵乘法的结合律，把复杂度从 $O(N^2)$ 降到 $O(N)$。

2.1 线性化推导

因果自注意力原始公式：

$$\mathbf{O} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\right)\mathbf{V}$$

只看第 i 个位置的 token，把 softmax 展开：

$$\mathbf{o}_i = \sum_{j=1}^{i} \frac{\exp(s_{ij})}{\sum_{j'=1}^{i} \exp(s_{ij'})} \mathbf{v}_j, \qquad s_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d}}$$

合并通分（分子、分母各自累加，共用同一个归一化分母）：

$$\mathbf{o}_i = \frac{\sum_{j=1}^{i} \exp(s_{ij})\, \mathbf{v}_j}{\sum_{j=1}^{i} \exp(s_{ij})}$$

s_{ij} 本来就是算 $\mathbf{q}_i$ 与 $\mathbf{k}_j$ 的相似度，标准化写成 $\mathrm{sim}(\mathbf{q}_i, \mathbf{k}_j)$：

$$\mathbf{o}_i = \frac{\sum_{j=1}^{i} \mathrm{sim}(\mathbf{q}_i, \mathbf{k}_j)\, \mathbf{v}_j}{\sum_{j=1}^{i} \mathrm{sim}(\mathbf{q}_i, \mathbf{k}_j)}$$

既然本意就是算相似度，换成别的相似度函数也行。这里抽象成核函数 $\mathrm{sim}(\mathbf{q}, \mathbf{k}) = \phi(\mathbf{q})^\top \phi(\mathbf{k})$：

$$\mathbf{o}_i = \frac{\sum_{j=1}^{i} \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j)\, \mathbf{v}_j}{\sum_{j=1}^{i} \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j)}$$

原来的 sim 把 $\mathbf{q}$ 和 $\mathbf{k}$ 绑在一起，拆成 $\phi(\mathbf{q})^\top \phi(\mathbf{k})$ 后就不再受这个限制。$\phi(\mathbf{q}_i)^\top$ 与 j 无关，可以从累加符号里提出来：

$$\mathbf{o}_i = \frac{\phi(\mathbf{q}_i)^\top \sum_{j=1}^{i} \phi(\mathbf{k}_j)\, \mathbf{v}_j^\top}{\phi(\mathbf{q}_i)^\top \sum_{j=1}^{i} \phi(\mathbf{k}_j)}$$

2.2 等效 RNN 递推

原先算 $\mathrm{sim}(\mathbf{q}, \mathbf{k})$ 时，每个位置的 $\mathbf{q}_i$ 不同，每一步都要拿当前 query 和所有历史 key 重新算一遍相似度。拆成 $\phi(\mathbf{q})^\top \phi(\mathbf{k})$ 之后，历史里每一项只由该位置的 $\mathbf{k}_j, \mathbf{v}_j$ 决定，与当前 $\mathbf{q}_i$ 无关，可以事先累好，不必重复计算。

记分子里的 KV 累加为 $\mathbf{S}$，分母里的 $\mathbf{K}$ 累加为 $\mathbf{Z}$。推理时只需把当前步的新 KV 加到上一步的 $\mathbf{S}, \mathbf{Z}$ 上：

$$\mathbf{S}_0 = \mathbf{0}, \qquad \mathbf{Z}_0 = \mathbf{0}$$

$$\mathbf{S}_i = \mathbf{S}_{i-1} + \phi(\mathbf{k}_i)\,\mathbf{v}_i^\top$$

$$\mathbf{Z}_i = \mathbf{Z}_{i-1} + \phi(\mathbf{k}_i)$$

$$\mathbf{o}_i = \frac{\phi(\mathbf{q}_i)^\top \mathbf{S}_i}{\phi(\mathbf{q}_i)^\top \mathbf{Z}_i}$$

线性注意力因此等价于一个 RNN：用固定大小的状态 $(\mathbf{S}_i, \mathbf{Z}_i)$ 压缩历史，每来一个 token 更新一次状态，再用当前 $\mathbf{q}_i$ 读出 $\mathbf{o}_i$——论文标题 Transformers are RNNs 说的就是这件事。

- $\mathbf{S}_i$ 是固定大小的**关联记忆矩阵**：历史 key–value 被压进这张表里。读出时，用当前 $\mathbf{q}_i$ 与表做内积 $\phi(\mathbf{q}_i)^\top \mathbf{S}_i$，相当于按 query 从记忆里取出相应内容；再除以 $\phi(\mathbf{q}_i)^\top \mathbf{Z}_i$ 做归一化，才得到 $\mathbf{o}_i$。
- $\mathbf{Z}_i$ 不是记忆本身，只是配合归一化的**累加量**——记着历史上 $\phi(\mathbf{k})$ 一共积了多少，好让分母有意义。

2.3 局限性

- 存在的问题：
  1. **写入只是累加：** 每来一个 token，就把 $\phi(\mathbf{k}_i)\mathbf{v}_i^\top$ 叠到 $\mathbf{S}$ 上；$\mathbf{Z}$ 也同样只往上加。
  2. **不能按误差纠错：** 不会先按当前 key 从表里读出旧答案，再按「差了多少」去改；同一模式重复出现只会越叠越强，挤不掉已有关联。
  3. **没有遗忘机制：** 旧记忆只增不减，无法主动淡掉整段不再需要的上下文。

3. DeltaNet 思想：改这张表时，改用 Delta Rule

论文：Linear Transformers Are Secretly Fast Weight Programmers（Schlag、Irie、Schmidhuber，ICML 2021）

3.1 与 1991 快权重的关联

§2 把因果注意力拆成线性形式、再写成 RNN 递推之后，回头对照 Schmidhuber（1991），会发现结构几乎一一对应。本篇论文把这种对应关系形式化论证，结论就是：**线性 Transformer 本质上就是快权重。**

这步主要是**认出了同一套数学对象**——S 从此有了「快权重 / 关联记忆」的名字，和 1991 年的叙事接上了。

同一套结构：1991 快权重 ↔ 2020 线性注意力

Learning to Control Fast-Weight Memories: An Alternative to Dynamic Recurrent Networks （Schmidhuber，FKI-147-91，1991）

概念	Schmidhuber 1991（快权重）	Linear Attention（§2）
双网络分工 谁学、谁变	慢网 **S** 学固定权重；快网 **F** 的权重 **W** 每步都变。	慢权重 $\mathbf{W_Q}$, $\mathbf{W_K}$, $\mathbf{W_V}$ 靠反向传播学；记忆矩阵 **S** 每 token 更新。
慢网每步输出 两路激活向量	FROM / TO 单元产生两路激活（Schlag 2021 记作 **a**, **b**）： $\mathbf{a}^{(t)},\ \mathbf{b}^{(t)} = W_a\mathbf{x}^{(t)},\ W_b\mathbf{x}^{(t)}$	投影出 key、value（读出还要 query）： $\mathbf{k}^{(i)},\ \mathbf{v}^{(i)},\ \mathbf{q}^{(i)} = W_K\mathbf{x}^{(i)},\ W_V\mathbf{x}^{(i)},\ W_Q\mathbf{x}^{(i)}$
外积写入 累加进记忆矩阵	两路激活做外积，叠到快权重上： $W^{(t)} = W^{(t-1)} + \mathbf{a}^{(t)} \otimes \mathbf{b}^{(t)}$	§2 拆核函数后，KV 正好也是外积累加： $\mathbf{S}_i = \mathbf{S}_{i-1} + \phi(\mathbf{k}_i)\,\mathbf{v}_i^\top$
从记忆中读出 矩阵 × 查询向量	用当前输入查快权重矩阵（同一时刻的 **x**）： $\mathbf{y}^{(t)} = W^{(t)}\mathbf{x}^{(t)}$	用 query 查表；**Z** 是归一化累加量： $\mathbf{o}_i = \frac{\phi(\mathbf{q}_i)^\top \mathbf{S}_i}{\phi(\mathbf{q}_i)^\top \mathbf{Z}_i}$
固定大小状态 历史压进一张表	快权重 $\mathbf{W^{(t)}}$ 维度固定，不随序列变长而膨胀。	$\mathbf{S_i}$、$\mathbf{Z_i}$ 同样是固定大小的 RNN 状态。

对照式按 Schlag et al. 2021 对两套符号的等价改写；1991 原文还没有 Q/K/V 的叫法。

慢权重（W_Q, W_K, W_V 等）跨 token 不变，靠反向传播来学；快权重 $\mathbf{S}$ 是当前序列里的工作记忆，来一个 token 更新一次。

认成快权重之后，论文接着分析 §2 那种**纯累加写入**有什么问题，能不能改。

3.2 写入：换成 Delta Rule

问题一：容量有限。 固定大小的 $\mathbf{S}$ 装不下无限多组 key–value。读出时靠 key 之间的内积区分记忆；key 落在 d 维空间里，彼此正交的向量最多 d 个。关联数超过这个量级，不同记忆会串在一起。

问题二：不能纠错。 累加写入见一次加一次，同一 key 重复出现只会越叠越强，没法按误差改掉旧关联。

不再无脑累加 KV，而是先读、再比、再改。先用当前 key 从表里读出旧答案：

$$\bar{\mathbf{v}}_i^\top = \phi(\mathbf{k}_i)^\top \mathbf{S}_{i-1}$$

再看和新 value 差了多少，按误差改表。论文还让每一步自己决定改多少，记作写入强度 $\beta_i \in (0,1)$：

$$\mathbf{S}_i = \mathbf{S}_{i-1} + \beta_i\, \phi(\mathbf{k}_i)\, (\mathbf{v}_i - \bar{\mathbf{v}}_i)^\top$$

- $\mathbf{v}_i$ 是这次要写成的内容，$\bar{\mathbf{v}}_i$ 是用当前 key 从表里读出的旧内容。差值 $\mathbf{v}_i - \bar{\mathbf{v}}_i$ 是还缺多少；
- 比如这次想写成「我喜欢草莓」，表里已经读出它的 80%。误差就是剩下的 20%，于是补这 20%，而不是再叠一整句 100%。

这就是把 §1 的 $\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\,(t-y)\,\boldsymbol{x}$ 搬到矩阵上：$\mathbf{S}$ 扮演 $\boldsymbol{w}$，$\phi(\mathbf{k}_i)$ 扮演输入，$\mathbf{v}_i$ 是要写入的目标，$\bar{\mathbf{v}}_i$ 是表里读出的预测。

把 $\bar{\mathbf{v}}_i^\top = \phi(\mathbf{k}_i)^\top \mathbf{S}_{i-1}$ 代回去：

$$\mathbf{S}_i = \mathbf{S}_{i-1} + \beta_i\, \phi(\mathbf{k}_i)\, \mathbf{v}_i^\top - \beta_i\, \phi(\mathbf{k}_i)\, \bar{\mathbf{v}}_i^\top$$

$$\mathbf{S}_i = \mathbf{S}_{i-1} + \beta_i\, \phi(\mathbf{k}_i)\, \mathbf{v}_i^\top - \beta_i\, \phi(\mathbf{k}_i)\phi(\mathbf{k}_i)^\top \mathbf{S}_{i-1}$$

$$\mathbf{S}_i = \left(\mathbf{I} - \beta_i\, \phi(\mathbf{k}_i)\phi(\mathbf{k}_i)^\top\right)\mathbf{S}_{i-1} + \beta_i\, \phi(\mathbf{k}_i)\, \mathbf{v}_i^\top$$

- 可以理解为：用k找出本次关联的记忆块儿，然后先删除它，在写入新的内容

先沿当前 key 抹掉一块旧关联，再写入新的 $\mathbf{v}_i$。

读出仍用当前 query 查表：

$$\mathbf{o}_i = \phi(\mathbf{q}_i)^\top \mathbf{S}_i$$

3.3 归一化：不用 $\mathbf{Z}$

§2 读出要除以 $\phi(\mathbf{q}_i)^\top \mathbf{Z}_i$，$\mathbf{Z}$ 记着历史上 $\phi(\mathbf{k})$ 的累加。但现在有两个问题：

- $\mathbf{Z}$ 每步只往上累加正数，序列一长容易数值爆炸；
- 对 Delta 的「写入 / 抹掉」两步也配不平。

论文换成了 **sum normalisation（求和归一化）**：不维护 $\mathbf{Z}$，而是在用 ϕ 之前，把向量各分量除以自身之和，让分量加起来等于 1。以 query 为例：

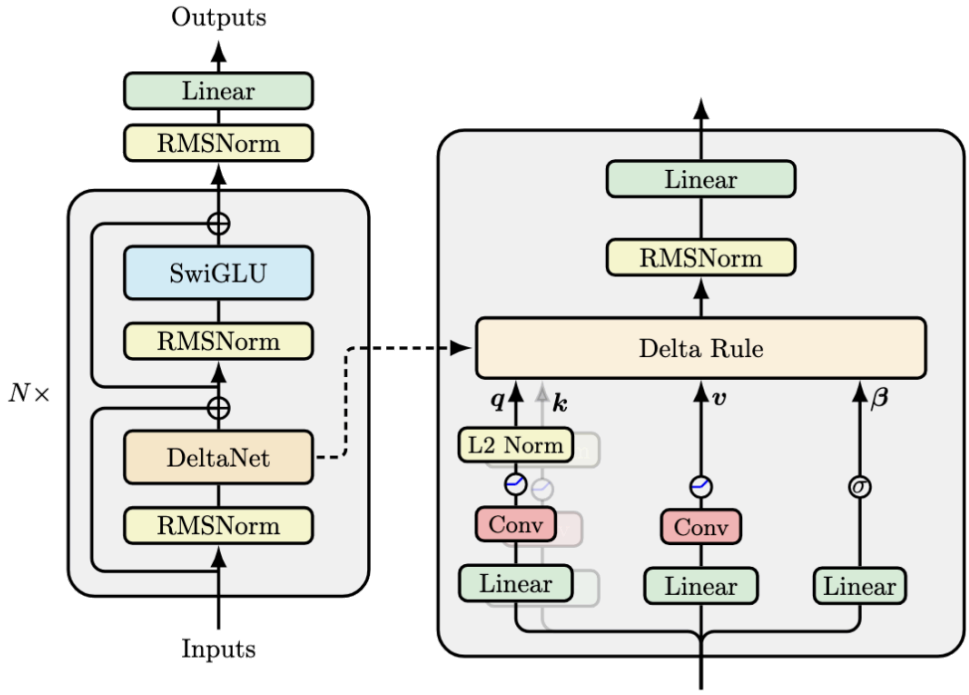
$$\phi'(\mathbf{q}^{(i)}) = \frac{\phi(\mathbf{q}^{(i)})}{\sum_{j=1}^{d_{\text{dot}}} \phi(\mathbf{q}^{(i)})_j}$$

$\phi(\mathbf{k})$ 同样处理。这样 $\phi(\mathbf{q})^\top \mathbf{S}$ 相当于对 $\mathbf{S}$ 的列做加权求和，权重和为 1，仍有「注意力」的语义。

3.4 把 DeltaNet 工程化（Yang 等，2024）

论文：Parallelizing Linear Transformers with the Delta Rule over Sequence Length（Yang、Wang 等，NeurIPS 2024）

§3.1–3.3 是 2021 的**写入规则**；这篇把 DeltaNet **装进现代 Transformer 式大模型**——宏观上仍是 Llama 那套（RMSNorm → token mixer → SwiGLU），只是把自注意力换成 DeltaNet 层，并定下一套沿用至今的 **Q/K/V 预处理 + 输出侧**配置。作者本人博客称这一步为 Modernize DeltaNet neural architecture。



进入 Delta Rule 之前，图右侧四条支路写成：

$$\begin{aligned}
\mathbf{q}_t &= \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_Q \mathbf{x}_t))) \\
\mathbf{k}_t &= \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_K \mathbf{x}_t))) \\
\mathbf{v}_t &= \mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_V \mathbf{x}_t)) \\
\beta_t &= \sigma(\mathbf{W}_\beta \mathbf{x}_t) \\
\mathbf{y}_t &= \mathbf{W}_O\, \mathrm{RMSNorm}(\mathrm{DeltaRule}(\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t, \beta_t))
\end{aligned}$$

- **Linear**：把当前 token $\mathbf{x}_t$ 投影成 q、k、v 或 β。
- **ShortConv**：一维卷积，可以让当前位置看到前面几个 token，增加一些局部信息。

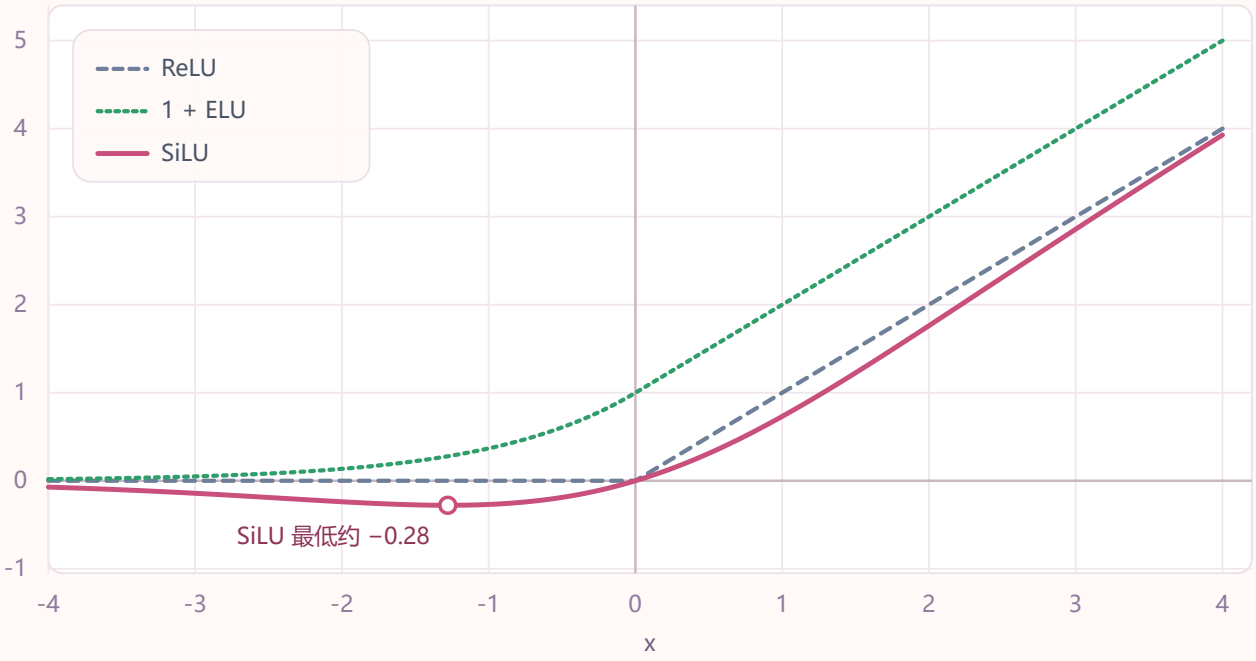
- **L2Norm**：只加在 q、k 上，把向量压成单位长度，保证沿当前 key 抹旧写新时数值稳定。
- β_t：Linear 再过 Sigmoid，得到 $(0,1)$ 的写入强度，不经过短卷积和 SiLU。
- **RMSNorm**：读出之后、输出投影之前做一次归一化，代替 §2 那个会越加越大的分母。

q、k、v 在短卷积之后的激活是 SiLU。正半轴接近 ReLU，负半轴有一个浅坑，不会把负数直接砍成 0。消融里它好于 ReLU 和当年的 $1+\mathrm{ELU}$。

$$\mathrm{ReLU}(x) = \max(0, x)$$

$$1 + \mathrm{ELU}(x) = \begin{cases} x + 1, & x > 0 \\ e^x, & x \le 0 \end{cases}$$

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$



3.5 层间混合

训到语言建模规模后，固定大小的 $\mathbf{S}$ 做精确长程检索仍不够，作者试了混合方案：

1. **隔一层插 SWA**（滑动窗口注意力）——约一半层看局部邻域；
2. **全网只留两层全局注意力**（靠前一层 + 中间附近一层），其余仍是 DeltaNet。

1.3B / 100B tokens 设定下，原文 Table 1 三列检索任务（越高越好）：

模型	SWDE	SQuAD	FDA
DeltaNet（纯线性）	49.5	37.4	17.2
+ Sliding Attn	53.3	43.3	22.3
+ Global Attn（2 层）	71.0	43.0	29.8

3.6 局限性

- 存在的问题：
  1. ~~**写入只是累加**：每来一个 token，就把 $\phi(\mathbf{k}_i)\mathbf{v}_i^\top$ 叠到 $\mathbf{S}$ 上；$\mathbf{Z}$ 也同样只往上加。~~
  2. ~~**不能按误差纠错**：不会先按当前 key 从表里读出旧答案，再按「差了多少」去改；同一模式重复出现只会越叠越强，挤不掉已有关联。~~
  3. **没有遗忘机制**：旧记忆只增不减，无法主动淡掉整段不再需要的上下文。

4. Gated DeltaNet：在 DeltaNet 上再加整体遗忘

论文：Songlin Yang、Jan Kautz、Ali Hatamizadeh，Gated Delta Networks: Improving Mamba2 with Delta Rule（ICLR 2025）

arXiv：https://arxiv.org/abs/2412.06464

理论线还留下一条：**没有整体遗忘**——表只会按 key 定点改，没法像 Mamba-2 / GLA 那样让整张表一起淡掉。上下文一切换，旧记忆还占着容量，真实任务里就弱。

本篇做的事很直接：在 Delta 改表**之前**，先给旧表乘一个门控 $\alpha_t \in (0,1)$。α_t 管「旧记忆整体留多少」，原来的 β_t 仍管「这个 key 改多少」。两条能力合在一起，就是 **Gated Delta Rule**。

4.1 两条线：整体淡 vs 定点改

把记忆表 $\mathbf{S}$ 想成一张关联表，两条常见策略各管一头：

Mamba-2 式门控（整体淡）。 每步先把整张表乘同一个标量，再叠上新的外积：

$$\mathbf{S}_t = \alpha_t\,\mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top, \qquad \alpha_t \in (0,1)$$

$\alpha_t \to 0$ 时旧表几乎清空，适合换上下文、清噪声；但**所有** key–value 同速淡，想忘掉某一格也会误伤别的格。

DeltaNet 式定点改（§3）。 先按当前 key 读旧答案，再按误差改这一格：

$$\mathbf{S}_t = \mathbf{S}_{t-1}\left(\mathbf{I} - \beta_t\,\mathbf{k}_t\mathbf{k}_t^\top\right) + \beta_t\,\mathbf{v}_t\mathbf{k}_t^\top$$

定点改得准，却**缺整体清表**：上下文一切换，旧关联还占着固定大小的 $\mathbf{S}$。

论文的观察是：这两种机制**互补**——门控擅长快速抹掉整段，Delta 擅长精确改某一格。合在一起，比单用哪一头都完整。

4.2 Gated Delta Rule

在 Delta 更新前先乘门控。写成矩阵形式（记号与 §3 一致，略去 ϕ）：

$$\mathbf{S}_t = \mathbf{S}_{t-1}\,\alpha_t\big(\mathbf{I} - \beta_t\,\mathbf{k}_t\mathbf{k}_t^\top\big) + \beta_t\,\mathbf{v}_t\mathbf{k}_t^\top$$

直觉拆两步：

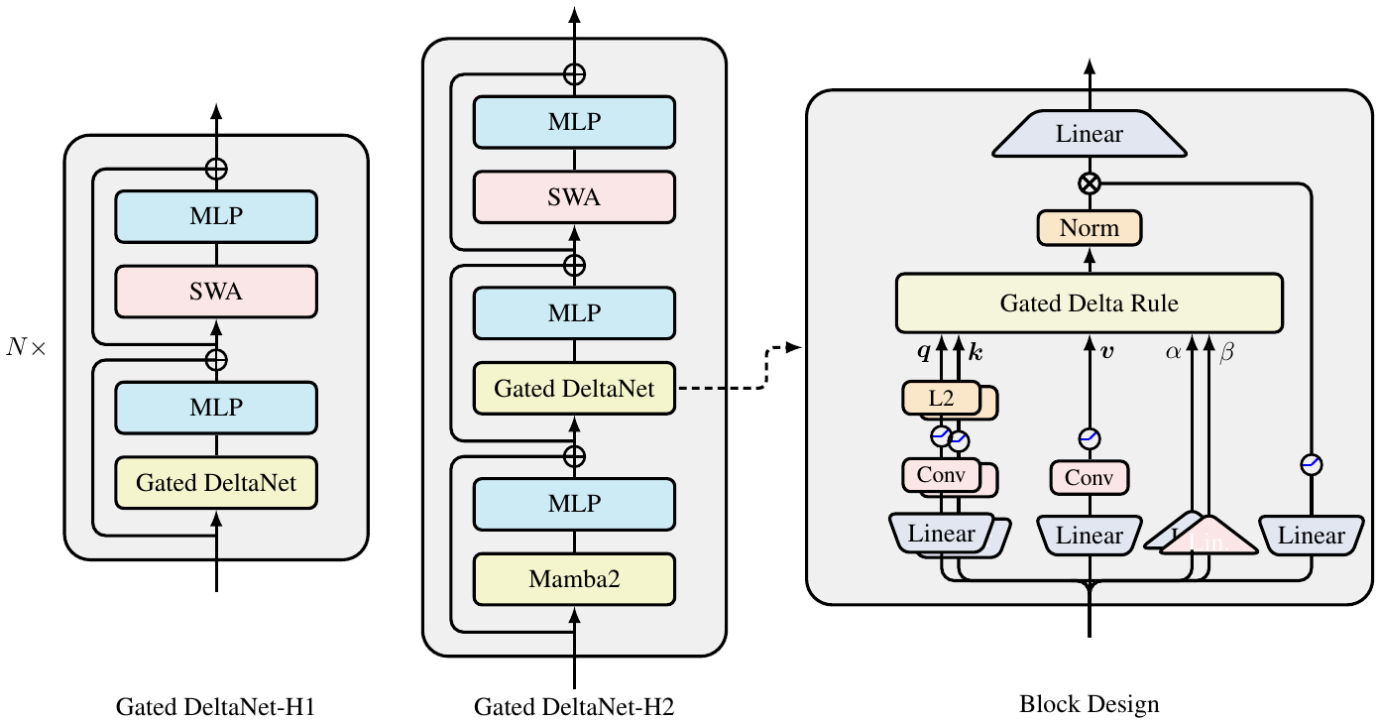
1. **整体衰减：** $\mathbf{S}_{t-1} \leftarrow \alpha_t \mathbf{S}_{t-1}$——整张表按 α_t 淡一档；
2. **定点 Delta：** 再对当前 key 做抹旧写新（强度仍是 β_t）。

两个极端：

- $\alpha_t \to 0$：旧表先被清掉，接近「重新开一张表再写」；
- $\alpha_t \to 1$：退回纯 DeltaNet（§3）。

4.3 模块配置（相对 §3.4）

宏观仍是 Llama 式堆叠；token mixer 换成 **Gated Delta Rule**。原文 Figure 1 整张如下（左 / 中为 H1、H2 混层，右为 Block Design）：



对照 §3.4，右侧 Block Design 各支路如下：

	DeltaNet（§3.4）	Gated DeltaNet
$\mathbf{q}_t$	$\mathbf{q}_t = \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_Q\mathbf{x}_t)))$	$\mathbf{q}_t = \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_Q\mathbf{x}_t)))$
$\mathbf{k}_t$	$\mathbf{k}_t = \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_K\mathbf{x}_t)))$	$\mathbf{k}_t = \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_K\mathbf{x}_t)))$
$\mathbf{v}_t$	$\mathbf{v}_t = \mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_V\mathbf{x}_t))$	$\mathbf{v}_t = \mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_V\mathbf{x}_t))$
α_t	—	$\alpha_t = \sigma(\mathbf{W}_\alpha\mathbf{x}_t)$
β_t	$\beta_t = \sigma(\mathbf{W}_\beta\mathbf{x}_t)$	$\beta_t = \sigma(\mathbf{W}_\beta\mathbf{x}_t)$
$\mathbf{y}_t$	$\mathbf{y}_t = \mathbf{W}_O\,\mathrm{RMSNorm}(\mathrm{DeltaRule}(\mathbf{q}_t,\mathbf{k}_t,\mathbf{v}_t,\beta_t))$	$\mathbf{y}_t = \mathbf{W}_O\,(\mathrm{SiLU}(\mathbf{W}_g\mathbf{x}_t)\odot\mathrm{Norm}(\mathrm{GatedDeltaRule}(\mathbf{q}_t,\mathbf{k}_t,\mathbf{v}_t,\alpha_t,\beta_t)))$

- α_t：得到 $(0,1)$ 的整体遗忘强度。

- **输出门**：再算一道 $\mathrm{SiLU}(\mathbf{W}_g\mathbf{x}_t)$，与 Norm 后的读出**逐元素相乘**（$\odot$），决定各通道往外传多少，最后才投影。

4.4 层间混合

论文在纯 Gated DeltaNet 之外，还试了两种 hybrid：

- **H1**：Gated DeltaNet 与 **SWA**（滑动窗口，约 2K）交错；
- **H2**：再叠进 **Mamba2**，顺序大致是 Mamba2 → Gated DeltaNet → SWA。

模型	SWDE	SQD	FDA
Gated DeltaNet（纯线性）	25.4	34.8	23.7
Gated DeltaNet-H1	35.6	39.7	52.0
Gated DeltaNet-H2	38.2	40.4	50.7

4.5 局限性

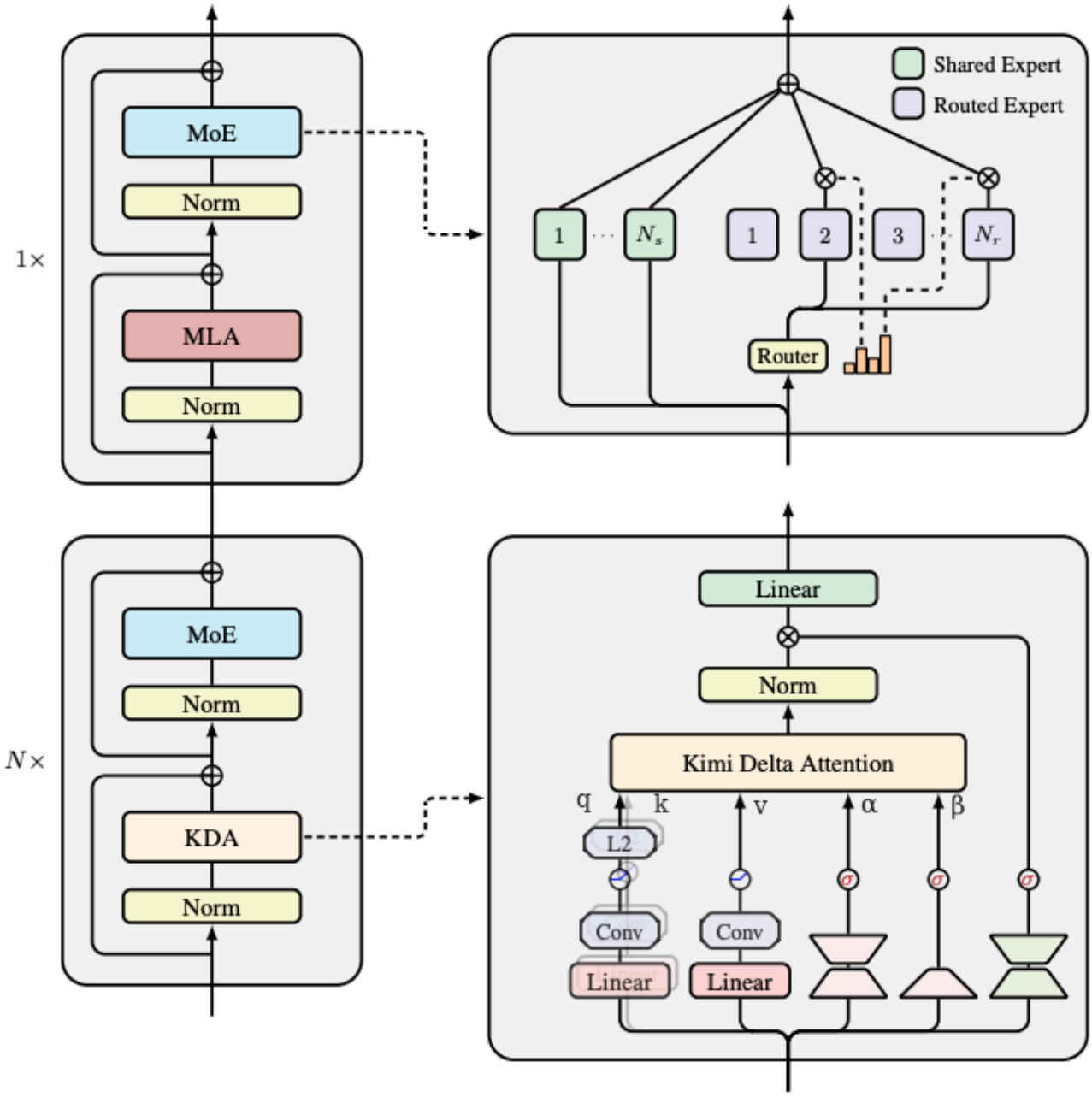
- 存在的问题：
  1. ~~**写入只是累加**~~
  2. ~~**不能按误差纠错**~~
  3. ~~**没有遗忘机制：**已用 α_t 做整体衰减。~~
  4. ~~**训大后仍缺整体遗忘**~~（门控已补上整表淡掉的能力）
  5. **遗忘太粗：一个 head 共用同一个 α_t。** 表的所有方向同速淡；有的通道该留、有的该忘，拆不开——下一节（§5 KDA）把标量门换成按通道的对角门。

5. Kimi Delta Attention：遗忘从「整张一起」变成「按通道」

- 论文：Kimi Team，Kimi Linear: An Expressive, Efficient Attention Architecture（2025）
- arXiv：https://arxiv.org/abs/2510.26692

5.1 模块配置（相对 §4）

原文 Figure 3 整张如下（左：KDA / MLA 与 MoE 的层叠；右上：MoE；右下：KDA block）：



	Gated DeltaNet (§4)	Kimi Delta Attention
$\mathbf{q}_t$	$\mathbf{q}_t = \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_Q\mathbf{x}_t)))$	$\mathbf{q}_t = \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_Q\mathbf{x}_t)))$
$\mathbf{k}_t$	$\mathbf{k}_t = \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_K\mathbf{x}_t)))$	$\mathbf{k}_t = \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_K\mathbf{x}_t)))$
$\mathbf{v}_t$	$\mathbf{v}_t = \mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_V\mathbf{x}_t))$	$\mathbf{v}_t = \mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_V\mathbf{x}_t))$
α_t / $\boldsymbol{\alpha}_t$	$\alpha_t = \sigma(\mathbf{W}_\alpha\mathbf{x}_t)$	$\boldsymbol{\alpha}_t = f(\mathbf{W}_\alpha^{\uparrow}\mathbf{W}_\alpha^{\downarrow}\mathbf{x}_t) \in [0,1]^{d_k}$
β_t	$\beta_t = \sigma(\mathbf{W}_\beta\mathbf{x}_t)$	$\beta_t = \sigma(\mathbf{W}_\beta\mathbf{x}_t)$
$\mathbf{y}_t$	$\mathbf{y}_t = \mathbf{W}_O\,(\mathrm{SiLU}(\mathbf{W}_g\mathbf{x}_t) \odot \mathrm{Norm}(\mathrm{GatedDeltaRule}(\mathbf{q}_t,\mathbf{k}_t,\mathbf{v}_t,\alpha_t,\beta_t)))$	$\mathbf{y}_t = \mathbf{W}_O\,(\sigma(\mathbf{W}_g^{\uparrow}\mathbf{W}_g^{\downarrow}\mathbf{x}_t) \odot \mathrm{RMSNorm}(\mathrm{KDA}(\mathbf{q}_t,\mathbf{k}_t,\mathbf{v}_t,\boldsymbol{\alpha}_t,\beta_t)))$

- $\boldsymbol{\alpha}_t$：从标量变成**通道向量**，做成 $\mathrm{Diag}(\boldsymbol{\alpha}_t)$。低秩投影后再经衰减函数 f 压到 $[0,1]$（细节见 §6）。
- **输出门**：仍是逐元素相乘决定往外传什么，但改成**低秩投影 + Sigmoid**。

5.2 层间混合

采用 **3 层 KDA : 1 层 MLA**（全局潜在注意力）均匀交错。

> 做过比例消融：3:1 验证集最好；7:1 训得动但验证差；1:1 质量接近却更贵。

MLA 不用位置编码（NoPE）： 位置与近因偏置交给 KDA（通道衰减本身就像可学习的乘性位置编码）。MLA 只做全局内容交互，省掉 RoPE 调频 / YaRN；推理时也更容易收成纯 MQA。消融里 NoPE 长文好于给 MLA 加 RoPE。

KDA:MLA	Training PPL ↓	Validation PPL ↓
0:1（全 MLA）	9.45	5.77
1:1	9.29	5.66
3:1	9.23	5.65
7:1	9.23	5.70
15:1	9.34	5.82

6. Kimi K3：把 KDA 用到超大模型上

- 论文：Kimi Team，Kimi K3: Open Frontier Intelligence（2026）
- arXiv：https://arxiv.org/abs/2607.24653

6.1 模块配置（相对 §5）

沿用 KDA 的基础上，做了以下几点优化：

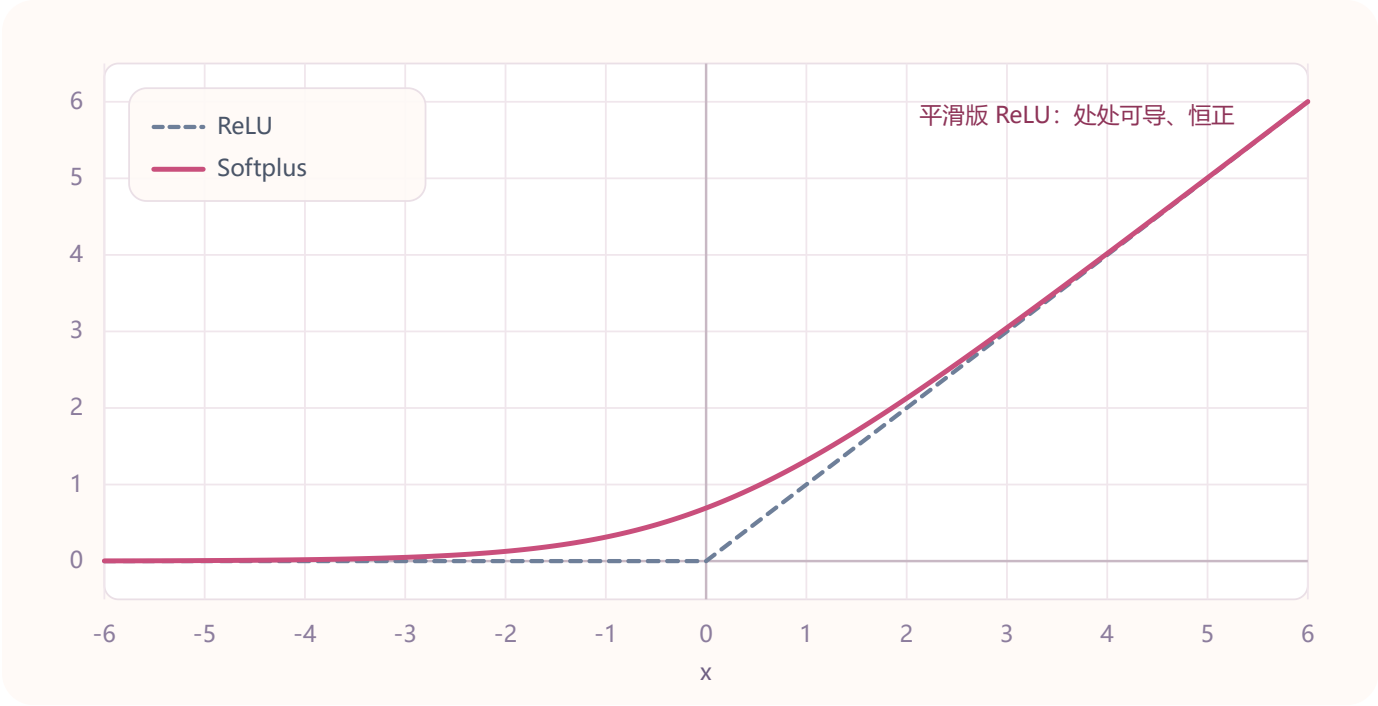
衰减加下界

上一章 KDA 的通道衰减 $\boldsymbol{\alpha}_t = f(\mathbf{W}_\alpha^{\uparrow}\mathbf{W}_\alpha^{\downarrow}\mathbf{x}_t)$ 是简略写法，具体公式如下。低秩投影先得到 $\mathbf{z}_t$，然后

$$\mathbf{g}_t = -e^{A}\,\mathrm{Softplus}(\mathbf{z}_t) \in (-\infty, 0), \qquad \boldsymbol{\alpha}_t = \exp(\mathbf{g}_t) \in (0,1)^{d_k}$$

这里用的 **Softplus** 是光滑的 ReLU：恒正、处处可导；大正数时接近 x，大负数时贴向 0。

$$\mathrm{Softplus}(x) = \log(1 + e^{x})$$



也就是 $f(\mathbf{z}) = \exp\big(-e^{A}\mathrm{Softplus}(\mathbf{z})\big)$。Softplus 越大 → $\mathbf{g}$ 越负 → $\boldsymbol{\alpha}$ 越接近 0（忘得越狠）。

问题在于 $\mathbf{g}$ **没有下界**，可以冲到 $-\infty$。chunk 并行要对累计衰减 $\boldsymbol{\Gamma}$ 做 $1/\boldsymbol{\Gamma}$，$\boldsymbol{\alpha}$ 一旦极小，BF16 里就容易炸；

于是把 log 衰减改成**有下界**的 scaled Sigmoid（$g_{\min} = -5$）：

$$\mathbf{g}_t = g_{\min}\,\sigma(e^{A_h}\mathbf{z}_t) \in (g_{\min}, 0), \qquad \boldsymbol{\alpha}_t = \exp(\mathbf{g}_t) \in (e^{g_{\min}}, 1)$$

取 -5 是为了对齐常见的 16-token 块：每步最狠也就忘到 e^{-5}，每一块累计 log 衰减落在 $(-80, 0)$，倒数小于 e^{80}，仍在 BF16 动态范围内。

全秩输出门

KDA 的输出门是低秩投影 + Sigmoid：$\sigma(\mathbf{W}_g^{\uparrow}\mathbf{W}_g^{\downarrow}\mathbf{x}_t) \odot \mathrm{RMSNorm}(\mathbf{o}_t)$。规模上来后，K3 把输出门改回**全秩** $\mathbf{W}_g$；全局层也加成 **Gated MLA**（同样 $\sigma(\mathbf{W}_g\mathbf{x})\odot$ 读出）。

两个改动对照如下

	Kimi Linear KDA（§5）	Kimi K3 中的 KDA
$\boldsymbol{\alpha}_t$	$\boldsymbol{\alpha}_t = \exp\big(-e^{A}\mathrm{Softplus}(\mathbf{z}_t)\big)$（无下界）	$\mathbf{g}_t = g_{\min}\sigma(e^{A_h}\mathbf{z}_t)$，$\boldsymbol{\alpha}_t = \exp(\mathbf{g}_t)$（$g_{\min} = -5$）
$\mathbf{y}_t$	$\mathbf{y}_t = \mathbf{W}_O\big(\sigma(\mathbf{W}_g^{\uparrow}\mathbf{W}_g^{\downarrow}\mathbf{x}_t) \odot \mathrm{RMSNorm}(\tilde{\mathbf{o}}_t)\big)$（低秩门）	$\mathbf{y}_t = \mathbf{W}_O\big(\sigma(\mathbf{W}_g\mathbf{x}_t) \odot \mathrm{RMSNorm}(\tilde{\mathbf{o}}_t)\big)$（全秩门）

6.2 层间混合

每块 `3×KDA + 1×Gated MLA`（3:1），骨干末尾再加一层 Gated MLA，保证最后一层是全局。约 93 层里大致是 69 层 KDA + 24 层 MLA。

为什么混： 沿用 Kimi Linear 已验证的 3:1；本文重点在规模、MoE、工程与百万级上下文，不再重开混层比例表。

MLA 仍是 NoPE： 与 §5.2 相同——位置与近因交给 KDA，MLA 只做全局内容交互；也避免拉长上下文时重调 RoPE / YaRN。